目录

RVV扩展指令综合分析报告

一、扩展指令方案汇总
二、热点分布与收益数据支撑
三、CX 指令延迟参考
四、BBV 实测数据
五、与主流架构对比

RVV扩展指令综合分析报告

分析范围：基于 llama.cpp、YOLO11n（FP32/INT8）、ResNet50 三个推理场景的 BBV profiling + perf 实测数据，

识别 RVV 指令集与主流架构（x86 VNNI/AVX-512、ARM NEON/UDOT/SMMLA、Power VSX MMA、LoongArch LSX、S390X、WASM SIMD）的差距，

提出硬件指令扩展需求并量化收益。

>

数据来源：

- llama.cpp perf: `applications/llama.cpp/README.md`（SpacemiT K1-X, Qwen2.5-0.5B Q4_K_M）+ `docs/report/llama.cpp/rvv-gap-analysis-consolidated-llama.cpp-2026-04-26.md`
- llama.cpp BBV: `output/bbv_rvv512/llama.cpp/`（QEMU BBV, VLEN=512, 4 算子独立测试），`docs/report/llama.cpp/rvv-gap-analysis-{gemm,gemv,quantize,vec-dot}-*.md`
- YOLO FP32 perf: `applications/onnxrt/yolo/data/perf/yolo11n_bananapi_k1_rv64gcv_scalar_20260424_analysis.md`（SpacemiT K1）
- YOLO INT8 perf: `applications/onnxrt/yolo/data/perf-int8-yolo11n/summary.md`（SpacemiT K1, RVV512-patched）
- YOLO FP32 BBV: `output/bbv_rvv512/sgemm/`（QEMU BBV profiling, MlasSgemmKernelRvv512）
- YOLO INT8 BBV: `output/bbv_rvv512/qgemm/`（QEMU BBV profiling, MlasQgemmKernelRvv512）
- YOLO INT8 算子 BBV: `output/bbv_rvv512/{compute-logistic,quick-gelu,reduce-minmax-f32,quantize-linear}/`
- ResNet50 SGEMM BBV: `docs/report/resnet/rvv-gap-analysis-sgemm-kernel-vl16-2026-04-25.md`
- ONNXRT SGEMM FP32 BBV: `docs/report/onnxrt/rvv-gap-analysis-sgemm-kernel-vl16-2026-04-26.md`
- OSTrack ReduceMean: `docs/report/onnxrt/ostrack/rvv-gap-analysis-reducemean-f32-2026-04-26.md`
- OSTrack Softmax: `docs/report/onnxrt/ostrack/rvv-gap-analysis-softmax-f32-2026-04-26.md`
- OSTrack 综合: `docs/report/onnxrt/ostrack/rvv-gap-analysis-ostrack-consolidated-2026-04-26.md`
- CX 指令延迟: `docs/reference/cx/instruction-constraints-and-latency.md`

一、扩展指令方案汇总

命名约定：本报告为 RISC-V RVV 扩展指令方案的权威参考。各应用缺口分析报告中相同概念的扩展方案已统一命名：

- `vdot.vv` / `vusdot.vv` / `vdot.s8` → vsegdot.vv（方案 1）
- `vusdot_lane.vv` / `vdot_lane` → vdot_lane.vx（方案 2）
- `vmatmul.fp32` → vmulacc.vv（方案 5）（概念统一：4×4 FP32 矩阵外积 FMA）
- `vfmacc.vv_lane` → 维持不变（方案 4）
- `vinterleave.v` / `vlseg2e8` → vunzip/vzip.vv（方案 8）
- `vnibunpack.vv` / `vlse_unpack8` / `vunpackn.v` → vnibunpack.vv（方案 15）
- `vfadd.red` / `vaddv_f32` → vfadd.red.vs（方案 16）
- `vfexp` / `vexp2ps` / `vexp.approx` → vfexp.v（方案 17）

1.1 方案总表

编号	指令名称	类别	解决的问题	跨应用收益
1	vsegdot.vv	INT8	4×i8→i32 分段点积（替代 vwmulu+vwaddu 两步序列）	YOLO11n INT8: 54%, llama.cpp: 15-25%
2	vdot_lane.vx	INT8	Lane-indexed dot 消除标量广播	llama.cpp: 15-25%
3	vnarrow_sat	INT8	饱和窄化 int32→int8（替代 2 步 vncvt）	YOLO11n INT8: ~0.3%
4	vfmacc.vv_lane	FP32	Lane-indexed FMA 消除标量加载，K 步间复用 A 元素	YOLO FP32: 29.1%, ResNet50: 22.86%, llama.cpp: 5-8%
5	vmulacc.vv	FP32	4×4 矩阵外积 FMA + 专用累加器释放 VR（ONNXRT SGEMM 中亦称 vmatmul.fp32）	YOLO FP32: 44.6%, ResNet50: 44.55%, ONNXRT SGEMM: 77.5%（K-loop BB）
6	vclamp.vf	FP32	单指令 clamp 替代 vfmax+vfmin（2→1）	YOLO INT8: ~1.0%
7	vfmax.red/vfmin.red	FP32	单指令水平 min/max 归约（替代 3 步序列）	YOLO INT8: <0.5%, OSTrack Softmax: 归约 BB 67%
8	vunzip/vzip.vv	数据重排	奇偶分离/合并（转置加速）	YOLO: 边际收益
9	vwmaccwev/wod.vv	数据重排	Even/Odd 分离 widening MAC 减少依赖链	llama.cpp: 3-5%
10	vfncvt_scale_x_f_w_i8	FP32→INT8	f32×scale→i8 一步窄化（替代 vfmul+vfncvt+vncvt 3 步）	quantize BB: 19.5%
11	vwmulred.vs	INT8	Widening multiply→reduce 融合（替代 vwmul+vwredsum 2 步）	llama.cpp vec-dot: 2.9%
12	vfabs_redmax	FP32	ABS+归约最大值融合（替代 vfabs+vfredmax+vfmv.f.s 3 步）	quantize BB: 19.5%
13	prefetch.v	内存	向量数据预取（填补 x86 prefetcht0 差距）	大矩阵 GEMM: 5-15%
14	vsignext.vx_mu	INT8	掩码符号扩展融合（替代 vmnand+vsub.mu 2 步）	llama.cpp vec-dot: 1.2%
15	vnibunpack.vv	数据重排	单指令 nibble 解包（替代 vand+vsrl 2 步）	llama.cpp gemm: 2-3%
16	vfadd.red.vs	FP32 归约	单指令水平求和归约（替代 vfmv+vfredusum+vfmv.f.s 3 步）	OSTrack ReduceMean+Softmax: 归约 BB 67%
17	vfexp.v	FP32 超越函数	向量指数指令（替代 ~28 条多项式指令）	OSTrack Softmax exp BB: ~96%

收益说明：所有收益均为 Amdahl 定律计算的整体推理加速百分比，结合 perf 实测函数占比 + BBV 指令级数据 + CX 指令延迟表。方案 1-9 基于 YOLO/ResNet50/llama.cpp 早期分析，方案 10-17 基于 llama.cpp BBV profiling（2026-04-26）和 ONNXRT SGEMM 缺口分析新增。

1.2 INT8 路径方案详解

方案 1: vsegdot.vv — INT8 分段点积

> 审查注（2026-04-26）：RVV Zvdot4a8i 扩展已提供 `vdot4au_vv`（quad-widening 4D dot product, uint8×uint8→uint32）。二者数学功能等价，但数据布局不同：Zvdot4a8i 要求输入打包为 uint32 字（4 字节/字），vsegdot.vv 要求未打包的字节向量。vsegdot.vv 可视为 Zvdot4a8i 的"未打包"变体。

指令定义：

```
vsegdot.vv vd, vs1, vs2, vm
  功能: vd[i] += Σ(j=0..3) vs1[4i+j] × vs2[4i+j] (uint8×uint8→uint32, 4路分段规约)
  等效于: vwmulu + vwaddu 两步序列合并为单指令
  关联: Zvdot4a8i 扩展的 vdot4au 提供类似功能 (quad-widening 4D dot product)
```

跨平台来源：x86 AVX512_VNNI `vpdpbusd`（单指令 4×uint8×int8→int32），ARM NEON `vudot.u8`

RVV 现状：

- INT8 MAC 需要 2 条指令：`vwmulu.vx`（uint8×uint8→uint16，延迟 4+5=9 周期）+ `vwaddu.wv`（uint16→uint32 累加，延迟 4 周期）
- 还需要额外的 `lbu` 标量加载（4 周期）和 `vle8.v` 向量加载（3 周期）
- 编译器频繁切换 SEW（e8→e16→e32），产生大量 `vsetvli` 开销（占 K-loop 16.8%）

当前周期分析（K-loop，per PackedK group = 4 K 元素 × 16 列）：

操作	指令数	单条延迟	总周期
加载 A（lbu）	4	4	16
加载 B（vle8.v）	4	3	12
扩展乘（vwmulu.vx）	4	9	36
扩展加（vwaddu.wv）	4	4	16
SEW 切换（vsetvli）	9	~1	~9
关键路径估计			~52

扩展后周期分析：

```
vle32.v v_a, (a) # 加载4个A元素打包为32位 3周期
vle8.v v_b, (b) # 加载64个B元素 (4K×16N)  3周期
vsegdot.vv vacc, v_a, v_b # 4×i8 dot→i32, 16列并行 ~7周期
```

操作	指令数	单条延迟	总周期
加载 A（vle32.v）	1	3	3
加载 B（vle8.v）	1	3	3
分段点积（vsegdot.vv）	1	~7	7
关键路径合计	3		13 周期

收益计算：

```
K-loop 关键路径: 52 → 13 周期
K-loop 加速比: 52/13 = 4.00
函数加速比 (K-loop 占 96.0%) : 1/(0.04 + 0.96/4.00) = 3.57

YOLO11n INT8 整体收益:
  QGEMM 占 74.51% (perf 实测)
```

```
  整体加速 = 1/(0.2549 + 0.7451/3.57) = 2.157 → 54%

llama.cpp 整体收益: 15-25% (理论分析, gemv_q4_0 + gemv_q8_0)
```

与 ARM SDOT 对比:

平台	INT8 GEMM 指令/4K 步	总周期
ARM SDOT	`ldr q_a` + `ldr q_b` + `udot`	~13
RVV 现状	4×lbu + 4×vle8 + 4×vwmulu + 4×vwaddu + 9×vsetvli	~52
RVV + vsegdot	1×vle32 + 1×vle8 + 1×vsegdot	~13

vsegdot.vv 可将 RVV INT8 GEMM 提升到与 ARM SDOT 相同的效率水平。

方案 2: vdot_lane.vx — INT8 Lane-indexed Dot

指令定义:

```
vdot_lane.vx vd, vs2, vs1, imm
  功能: 从 vs1 提取第 imm 个元素广播, 与 vs2 逐元素 widening MAC
  等效于: 消除 vwmulu.vx 的标量广播开销 (4+5=9 周期 → ~7 周期)
```

跨平台来源: ARM NEON `vmla_lane` 系列

应用场景: llama.cpp INT8 GEMV 中标量 A 元素广播到向量 B

收益计算:

```
K-loop 级收益: 周期从 16→10, 加速 1.60
函数级收益 (K-loop 占 85%) : 加速 1.44
llama.cpp 整体: gemv_q4_0(40%) + gemv_q8_0(23%) 加速 1.44
  新时间 = 37s + 40s/1.44 + 23s/1.44 = 81s
  整体加速 ≈ 19% (范围 15-25%)
```

方案 3: vnarrow_sat — 饱和窄化

指令定义:

```
vnarrow_sat_i32_i8 vd, vs2 # int32→int8 饱和窄化 (1 条替代 2 步 vncvt)
vnarrow_sat_u32_u8 vd, vs2 # uint32→uint8 饱和窄化
vnarrow_sat_i32_i16 vd, vs2 # int32→int16 饱和窄化
```

跨平台来源: ARM NEON `vqmovn_s32`/`vqmovun_s16`, x86 SSE2 `_mm_packs_epi32`/`_mm_packus_epi16`

RVV 现状:

- U8 输出需 2 步窄化: `vncvt i32→i16` (4 周期) + `vncvt i16→i8` (4 周期)
- `vncvt_x_x_w` 是截断窄化 (无饱和保证), 需预先在 float 域 clamp

收益计算:

```
BBV 数据 (quantize-linear, VLEN=512) : 主循环 10 条指令/32 元素
CX 延迟: 当前窄化 8 周期 → 扩展后 4 周期
BB 总周期: ~43 → ~39, BB 内减少 ~9.3%
整体收益: 9.3% × 2.94% (QuantizeLinear 占比)  ≈ 0.3%
```

注: QuantizeLinear 占比小 (2.94%), 整体收益有限。主要价值在于提升鲁棒性 (消除截断溢出风险)。

1.2 附加方案

方案 11: vwmulred.vs — Widening Multiply-Reduce 融合

指令定义:

```
vwmulred.vs vd, vs2, vs1
  功能: vd[0] += Σ(vs2[i] × vs1[i]) // widening i8×i8→i16, sum→i32
  等效于: vwmul.vv + vwredsum.vs 两步融合为单指令
```

跨平台来源：ARM NEON `vdotq_s32` + `vaddvq_s32` 组合，WASM SIMD `i32x4.dot_i16x8` + horizontal add

RVV 现状：

- int8 dot product 需 2 步：`vwmul.vv`（widening multiply，4 周期）+ `vwredsum.vs`（归约，4 周期）= 8 周期
- 额外需要 `vmv.v.x`（清零累加器，3 周期）+ `vmv.x.s`（提取标量，3 周期）

收益计算：

```
当前 RVV (vec-dot-q5_0 核心路径)：
  vwmul.vv (4c) + vwredsum.vs (4c) + vmv.v.x (3c) + vmv.x.s (3c) = 14c

扩展后：
  vwmulred.vs (~6c) + vmv.x.s (3c) = 9c
  周期减少：14 → 9，加速 1.56

llama.cpp 整体 (vec-dot 占 54.64%)：
  整体加速：1/(0.4536 + 0.5464/1.56) = 1.243 → 内核级 +25%
  考虑 pipeline 效率 80%：整体推理约 +2.9%
```

对比 vsegdot.vv（方案 1）：vwmulred.vs 是更轻量的替代方案——仅融合 multiply+reduce 而不改变 SEW（无需 e8→e32 切换），但收益也较小。适合作为 vsegdot.vv 实现前的过渡方案。

方案 14: vsignext.vx_mu — 掩码符号扩展融合

指令定义：

```
vsignext.vx_mu vd, vm, vs2, rs1
  功能：vd[i] = vm[i] ? vs2[i] : (vs2[i] - rs1)
  等效于：vmnand + vsub.vx_mu 两步融合 (用于 Q5_0 符号扩展)
```

跨平台来源：ARM NEON `vsubq_s8`（查表后），x86 AVX2 `VPANDNOT` + `VPOR` + `VPSUBB`（3 步位扩展序列）

收益计算：

```
当前 RVV (vec-dot-q5_0 符号扩展)：
  vmnand.mm (3c) + vsub.vx_mu (7c) = 10c

扩展后：
  vsignext.vx_mu (~7c)
  周期减少：10 → 7，加速 1.43

llama.cpp 整体 (vec-dot 占 54.64%，符号扩展占内核 ~20%)：
  内核加速：1/(0.80 + 0.20/1.43) = 1.066
  整体加速：1/(0.4536 + 0.5464/1.066) = 1.035 → 整体推理约 +1.2%
```

注：整体收益有限（~1.2%），Q5_0/Q5_1/Q5_K 等 5-bit 量化类型均可受益。

1.3 FP32 路径方案详解

方案 4: vfmacc.vv_lane — Lane-indexed FMA

指令定义：

```
vfmacc.vv_lane vd, vs1, vs2, imm[5-bit]
  功能：vd[i] += vs2[i] × vs1[imm] (i = 0..VL-1)
        广播 vs1 的第 imm 个元素，与 vs2 逐元素乘加
```

跨平台来源（6 平台共识）：

平台	对应指令	映射关系
ARM NEON	`fmla v0.4s, v4.4s, v8.s[lane]`	最灵活：A 元素保持在向量寄存器，K 步间复用
x86 AVX-512F	`vfmadd231pf_bcst zmm, zmm, [mem]{1to16}`	最激进：内存加载+广播+FMA 合并为单指令

平台	对应指令	映射关系
S390X Z-Vector	`vec_perm` 广播设置	批量广播
LoongArch LASX	`xvldrepl.w`	内存加载+广播
Power VSX	`vmERGEh/vmERGEo` + `xvpermdi`	元素广播组合
RVV 现状	`vfmacc.vf`	仅接受 f 标量寄存器，无向量 lane 索引提取能力

BBV 实测数据（FP32 SGEMM K-loop，VLEN=512）：

K-loop 占函数指令执行：YOLO 94.68%，ResNet50 91.43%

K-loop 体（16 条指令/迭代）：

```
flw fa5,0(a4) # A 标量加载（行0，K=0）  4 周期
flw fa4,4(a4) # A 标量加载（行0，K=1）  4 周期
vle32.v v11,(t5) # B 向量加载（16 列，K=0）  3 周期
flw fa3,0(t4) # A 标量加载（行1，K=0）  4 周期
flw fa2,4(t4) # A 标量加载（行1，K=1）  4 周期
addi s0,t5,64 # 指针计算 1 周期
vfmacc.vf v9,fa5,v11 # FMA: acc_r0 += a0 * b 5 周期
vfmadd.vf v11,fa3,v10 # FMA: tmp += a1 * b 5 周期
vle32.v v10,(s0) # B 向量加载（16 列，K=1）  3 周期
addi a4,a4,8 # A 指针步进 1 周期
addi t5,t5,128 # B 指针步进 1 周期
addi t6,t6,-2 # K 计数器 1 周期
vfmacc.vf v9,fa4,v10 # FMA: acc_r0 += a1 * b 5 周期
vfmadd.vf v10,fa2,v11 # FMA: acc_r1 += a1 * b 5 周期
addi t4,t4,8 # A 指针步进 1 周期
bgtu t6,t3,-54 # 循环分支 1 周期
```

当前关键路径周期：4×flw(4) + 2×vle32(3) + 4×vfmacc(5) = 42 周期

扩展后：

```
vle32.v v_a, (a) # 1 次向量加载 4 个 A 元素 3 周期
vle32.v v_b0, (b) # B 向量加载 3 周期
vfmacc.vv_lane v_acc_r0, v_a, v_b0, 0 # lane-indexed FMA 5 周期
vfmacc.vv_lane v_acc_r1, v_a, v_b0, 2 # lane-indexed FMA 5 周期
vle32.v v_b1, (b+16) # B 向量加载 3 周期
vfmacc.vv_lane v_acc_r0, v_a, v_b1, 1 # lane-indexed FMA 5 周期
vfmacc.vv_lane v_acc_r1, v_a, v_b1, 3 # lane-indexed FMA 5 周期
```

扩展后关键路径周期：1×vle32(3) + 2×vle32(3) + 4×vfmacc_lane(5) = 29 周期

收益来源：4 次 `flw`（4×4=16 周期）→ 1 次 `vle32.v`（3 周期），节省 13 周期

各应用收益计算：

```
YOLO11n FP32:
  K-loop 加速: 42/29 = 1.448
  函数加速（K-loop 94.8%）: 1/(0.052 + 0.948/1.448) = 1.415
  整体加速（GEMM 76.86%）: 1/(0.2314 + 0.7686/1.415) = 1.291 → 29.1%

ResNet50:
  BB 内减少: (16-12)/16 = 25%
  整体收益: 25% × 91.43% = 22.86%

llama.cpp: 辅助优化，整体 5-8%
```

方案 5: vmulacc.vv — 4×4 矩阵外积 FMA

指令定义：

```
vmulacc.vv acc, vs2, vs1, sew=32
  功能: acc[4×4 矩阵] += vs2[0..3] × vs1[0..3]^T（外积）
  acc: 4×4 float32 累加块，独立于向量寄存器文件
```

```
  vs2: 4 个 float32 (列向量, 代表 A 的 4 行)
  vs1: 4 个 float32 (行向量, 代表 B 的 4 列)

  配套指令:
    vzero.acc acc — 清零累加块
    vread.acc vd, acc — 读取累加块到向量寄存器
    vwrite.acc acc, vs2— 写入向量寄存器到累加块
```

跨平台来源：Power VSX (POWER10) `xvf32gerpp` — 4x4 外积 MAC 写入专用累加器 `__vector_quad`（独立于 VR 寄存器文件）

核心收益：

1. 消除 per-element A 标量加载（4 行只需 1 次 A 加载，而非 16 次 flw）
2. 释放累加器 VR 寄存器（专用累加器不占用 VR），允许更宽 tile
3. 减少依赖链（4 条 MMA vs 16 条 vfmacc.vf per K-step）

各应用收益计算：

```
YOLO11n FP32:
  周期减少: 80→48, 加速 1.67
  整体加速 (GEMM 76.86%) : 1/(0.2314 + 0.7686/1.67) = 1.446 → 44.6%

ResNet50 (8 行 tile 场景) :
  BB 内减少: 48.8%
  整体收益: 48.8% × 91.43% = 44.55%

ONNXRT SGEMM (亦称 vmatmul.fp32) :
  K-loop 64 MACs 从 20 条指令 → 4 条, BB 内减少 80%
  整体收益: 80% × 96.9% (K-loop 占比)  ≈ 77.5% (K-loop BB 内)
```

方案 12: vfabs_redmax — ABS + 归约最大值融合

指令定义：

```
vfabs_redmax vd, vs2, vs1, vm
  功能: vd[0] = max(|vs2[0]|, |vs2[1]|, ..., |vs2[VL-1]|, vs1[0])
  等效于: vfabs + vfredmax + vfmv.f.s 三步融合
```

跨平台来源：ARM NEON `vmaxvq_f32`（仅归约+提取，无 ABS 融合），x86 亦需分步

应用场景：量化中 amax = max(|x|) 计算（所有量化 kernel）

收益计算：

```
当前 RVV (quantize-q8_0 amax 计算) :
  vfabs (3c) + vfredmax (4c) + vfmv.f.s (3c) = 10c, 3 条

扩展后:
  vfabs_redmax (~5c), 1 条
  BB 内减少: 每行节省 2 条, 每块(4行)节省 8 条
  BB 内百分比: 8/41 ≈ 19.5% (quantize-q8_0-4x4)
```

注：整体收益取决于 quantize 在推理中的占比（数据预处理阶段，非推理热点）。主要价值在于软件简化。

方案 6: vclamp.vf — 单指令 Clamp

指令定义：

```
vclamp.vf vd, vs2, min, max
  功能: vd[i] = clamp(vs2[i], min, max)
        等效于 max(min(vs2[i], max), min)
```

跨平台来源：各平台均需 2 条指令实现 clamp（vfmax+vfmin），无单指令方案。本方案填补所有平台的通用需求。

应用场景：

- Logistic/Sigmoid 激活：输入 clamp 到 [-18, 18] + 输出 clamp 到 [0, 1]，每 VL 迭代 2 次 clamp
- 其他需要饱和限幅的激活函数

收益计算：

```
BBV 数据（compute-logistic, VLEN=512）：核心循环 25 条指令/迭代，9,050 次执行
CX 延迟：当前 clamp 4 条(vfmax×2+vfmin×2) = 12 周期 → 扩展后 2 条 = 6 周期
BB 总周期（~57）减少 6 → ~10.5%
整体收益：10.5% × 9.80%（Logistic 占 YOLO INT8） ≈ 1.0%
```

方案 7: vfmax.red/vfmin.red — 单指令水平 Min/Max 归约

审查注（2026-04-26）：RVV 已有 `vfredmax.vs` / `vfredmin.vs` 提供单指令水平 max/min 归约。当前路径需 3 步（vfmv.v.f 初始化 + vfredmax/vfredmin + vfmv.f.s 提取标量），方案 7 将其简化为 1 步（隐式初始化 + 直接标量输出）。差距在 API 开销（3 指令 → 1 指令），非新操作。对标 ARM NEON64 `vmaxvq_f32`（单指令 + 标量输出）。

指令定义：

```
vfmax.red.vs vd, vs2 # vd[0] ← horizontal_max(vs2[0..VL-1])
vfmin.red.vs vd, vs2 # vd[0] ← horizontal_min(vs2[0..VL-1])
```

跨平台来源：ARM NEON64 `vmaxvq_f32`/`vminvq_f32`（单指令水平归约）

RVV 现状：

```
# 当前 3 步序列：
vfmv.v.f v_init, FLT_MAX, 1 # 3 周期（初始化）
vfredmin.vs v_red, v_acc, v_init, 16 # 4 周期（水平归约）
vfmv.f.s f_result, v_red # 3 周期（提取标量）
# 3 条指令，10 周期

# 扩展后：
vfmin.red.vs v_result, v_acc # ~4 周期
# 1 条指令，4 周期
```

收益计算：

```
BBV 数据（reduce-minmax-f32）：归约阶段仅执行 41 次 vs 主循环 3,114 次
归约阶段占函数：~1.3%
整体收益：4.63%（ReduceMinMax 占比）× 1.3% × 67% < 0.1%

注：主要价值在于软件简化和微架构优化，非显著性能收益。
ReduceMinMax 主循环已是 compute-bound（vfmin×4 + vfmax×4），归约不是瓶颈。
```

OSTrack Softmax Pass 1 应用（2026-04-26 新增）：Softmax 最大值查找（Pass 1）每 VL 迭代执行 1 次 max 归约。OSTrack-256 每次推理 144 次 Softmax × N=320，归约开销不可忽略。vfmax.red 单指令版本使归约指令从 3 → 1（67% 减少）。

方案 16: vfadd.red.vs — 单指令水平求和归约

审查注（2026-04-26）：RVV 已有 `vfredusum.vs`（无序）/ `vfredosum.vs`（有序）提供单指令水平 float sum 归约。`vfredusum` 语义：`vd[0] = sum(vs2[0..VL-1]) + vs1[0]`——与方案 16 的数学操作完全相同。当前路径需 3 步（vfmv.v.f 初始化零 + vfredusum + vfmv.f.s 提取标量 = 10 周期），方案 16 简化为 1 步（隐式零初始化 + 直接标量输出 = ~4 周期）。结论：归约指令已存在，差距在 API 开销（3 指令 → 1 指令）。 对标 ARM NEON64 `vaddvq_f32`（单指令 + 标量输出）。

指令定义：

```
vfadd.red.vs vd, vs2, vm
  功能：vd[0] ← Σ(vs2[0..VL-1]) // 水平求和归约到标量
  等效于：vfmv.v.f + vfredusum.vs + vfmv.f.s 三步融合为单指令
```

跨平台来源：ARM NEON64 `vaddvq_f32`（单指令 4→1 水平求和）

RVV 现状：

```
# 当前 3 步序列:
vfmv.v.f v_init, 0.0f, 1 # 3 周期 (初始化)
vfredusum.vs v_red, v_acc, v_init # 4 周期 (无序归约)
vfmv.f.s f_result, v_red # 3 周期 (提取标量)
# 3 条指令, 10 周期

# 扩展后:
vfadd.red.vs v_result, v_acc # ~4 周期
# 1 条指令, 4 周期
```

应用场景:

- ReduceMean sum 归约 (OSTrack: 50 次/推理, N=768)
- Softmax sum 归约 (OSTrack: 144 次/推理, N=320)
- LayerNorm 均值/方差计算
- 任何需要向量 → 标量 sum 归约的场景

收益计算:

```
归约 BB 内: 3 条 → 1 条, 减少 67%
OSTrack 单次推理 (194 次归约) : 节省 ~388 条指令
跨应用影响 (ReduceMean + Softmax) :
  ReduceMean: 每次归约节省 2 条, 50 次/推理
  Softmax: sum 归约阶段节省 2 条, 144 次/推理
```

注: 与方案 7 (vfmax.red/vfmin.red) 互补——方案 7 覆盖 max/min 归约, 方案 16 覆盖 sum 归约。二者共同实现归约操作的"单指令化"。

1.3 附加方案 (续)

方案 17: vfexp.v — 硬件向量指数指令

指令定义:

```
vfexp.v vd, vs2, vm
  功能: vd[i] ← exp(vs2[i]) // 向量指数函数 (有限精度硬件近似)
  精度: ~23-bit mantissa (与 x86 AVX512 ER vexp2ps 精度相当)
```

跨平台来源: x86 AVX512 ER `vexp2ps` (Xeon Phi) , ARM SVE 部分实现 `vexpa`

RVV 现状:

```
当前 exp(x) 计算 (sigmoid-based 多项式, ~28 条/VL) :
  vfmax (钳位) → vfmul (x²) → 22 条多项式 (p(x)+q(x) Horner 形式)
  → vfdiv (p/q) → vfadd (+0.5) → vfrsub (1-sig)
  → vfdiv (sig/(1-sig))
  总: ~28 条指令/VL 个元素

扩展后:
  vfexp.v vd, vs2 # 1 条指令/VL 个元素
  BB 内减少: ~96% (28 → 1)
```

收益计算:

```
OSTrack Softmax Pass 2 (占 softmax ~81% 指令) :
  exp 计算: ~1,100 条/VL 迭代 (N=320)  → ~40 条
  Pass 2 整体减少: ~63%
  整体 softmax 减少: ~63% (P2 单独)

跨应用影响:
  - Softmax (所有 transformer 模型)
  - GELU/Gaussian 激活函数
  - ComputeLogistic (sigmoid = 1/(1+exp(-x)))
```

注: `vfrsqrt7.v` / `vfrec7.v` 已在 RVV 中提供 7-bit 精度近似倒数/倒数平方根。`vfexp.v` 可参照此模式提供有限精度指数近似 (需 Newton-Raphson 精修步达到全精度) 。

方案 8: vunzip/vzip.vv — 奇偶分离/合并

指令定义：

```
vunzip.vv vd_even, vd_odd, vs2
  功能：将 vs2 奇偶元素分离
  输出：vd_even = [e0, e2, e4, ...], vd_odd = [e1, e3, e5, ...]

vzip.vv vd, vs1_even, vs2_odd
  功能：将两向量交错合并
  输出：vd = [e0, o0, e1, o1, ...]
```

应用场景：矩阵 4×4 转置、INT8 PackB 预处理

收益：转置函数级 25-40%，INT8 PackB 函数级 100-300%。对 YOLO 整体边际收益（非热点，占比 <5%）。

方案 9: vwmaccwev/wod.vv — Even/Odd Widening MAC

指令定义：

```
vwmaccwev.vv vd, vs2, vs1 ; even positions: vd.h[j] += vs2.b[2j] × vs1.b[2j]
vwmaccwod.vv vd, vs2, vs1 ; odd positions: vd.h[j] += vs2.b[2j+1] × vs1.b[2j+1]
```

跨平台来源：LoongArch LSX `vmaddwev_w_h`/`vmaddwod_w_h`

应用场景：llama.cpp INT8 GEMV widening MAC，减少依赖链深度

收益：llama.cpp 整体 3-5%

方案 15: vnibunpack.vv — 单指令 Nibble 解包

指令定义：

```
vnibunpack.vv vd_lo, vd_hi, vs2
  功能：vd_lo[i] = vs2[i] & 0xF; vd_hi[i] = vs2[i] >> 4
  等效于：vand.vx + vsrl.vx 两步融合（一步提取两个 nibble）
```

跨平台来源：x86 AVX2 `_mm256_and_si256` + `_mm256_srli_epi16` pattern，ARM NEON `vandq_u8` + `vshrq_n_u8` pattern

应用场景：所有 Q4_K/Q5_0/Q5_K 量化格式的 4-bit 权重解包

收益计算：

```
当前 RVV:
  vand.vx (3c) + vsrl.vx (3c) = 6c, 2 条
扩展后:
  vnibunpack (~4c), 1 条
  BB 内减少：每 K-loop 迭代节省 2 条 × 2-4 次 = 4-8 条
  llama.cpp gemm BB 内减少：~10%
  整体推理约 +2-3%
```

1.6 内存/预取方案详解

方案 13: prefetch.v — 向量数据预取

指令定义：

```
prefetch.v rs1, hint
  功能：将 rs1 指向的向量数据预取到 L1/L2 cache
  hint 编码预取策略（temporal/non-temporal, L1/L2/L3）
```

跨平台来源：x86 AVX `prefetcht0/prefetcht1/prefetcht2/prefetchnta` 系列，ARM `PRFM` 指令

应用场景：大矩阵 GEMM 中预取下一 K 行的 B 矩阵数据，减少 cache miss 延迟

RVV 现状：完全缺失。x86 SGEMM 在 K 循环中插入 `prefetcht0 [rdx+256]` 预取下一行数据，RVV 依赖硬件预取器。

收益：大矩阵 GEMM 场景下 5-15% 整体性能提升（基于 x86 预取优化经验）。对小矩阵和受限于计算而非内存的场景收益有限。

1.5 已排除方案

方案	排除原因
掩码存储优化	RVV 已有 `vse32.v + mask`，当前实现未使用。零 ISA 成本，纯软件修复
多行处理扩展	纯软件优化，将 SGEMM 行数从 2 扩展至 4/8 行，B 加载分摊。不需要新指令
配对向量加载	VLEN=512 下单条 `vle32.v` 已覆盖 16 列，配对加载收益 <1%
vfrsqrt7/vfrec7	RVV base V 扩展已包含，非新指令提议。当前实现未使用因精度需求不匹配
vexp.approx	已升级为正式方案 17 `vfexp.v`（OSTrack Softmax 分析提供量化收益：exp BB 减少 ~96%）

二、热点分布与收益数据支撑

2.1 llama.cpp 推理热点

来源：SpacemiT K1-X (rv64imafdcv)，Qwen2.5-0.5B Q4_K_M，128 tokens 生成，perf record

函数	占比	计算类型
`ggml_vec_dot_q5_0_q8_0_generic`	54.64%	Q5_0×Q8_0 向量点积
`ggml_vec_dot_q6_K_q8_K_generic`	18.19%	Q6_K×Q8_K 向量点积
`ggml_vec_dot_q4_K_q8_K_generic`	15.88%	Q4_K×Q8_K 向量点积
`ggml_gemv_q4_0_16x1_q8_0`	部分计入 vec-dot	INT8 量化 GEMV（decode 阶段）
`ggml_gemm_q4_0_16x1_q8_0`	部分计入 vec-dot	INT8 量化 GEMM（prefill 阶段）
`repack_q4_0_to_q4_0_16_bl`	~5%	数据重打包
其他算子	~6%	Norm/RoPE/Attention 等

量化点积总计：88.71%（vec-dot-q5_0 + vec-dot-q6_K + vec-dot-q4_K）

关键洞察：三个量化点积函数共享相同的 int8 MAC→int32 累加模式，单条 `vsegdot.vv`（方案 1）可同时加速全部 88.71% 的热点路径。gemv-q4_K kernel 因 LLVM 22 后端优化器 bug 当前为标量回退（仅 2 字节 trampoline），修复后可额外获得 +20-30% 推理加速。

2.2 YOLO11n 推理热点

FP32（vanilla ORT，SpacemiT K1，30 iters）：

函数	占比	计算类型
`MlasSgemmKernel<false,true>`	43.99%	FP32 标量 GEMM kernel
`MlasSgemmKernel<true,true>`	32.87%	FP32 标量 GEMM kernel
`MlasComputeLogistic`	8.28%	sigmoid 激活
`QuickGelu<float>::Compute`	0.01%	GELU fusion
`MlasConvIm2Col`	3.78%	im2col 变换

GEMM 总计：76.86%

INT8（RVV512-patched，SpacemiT K1，30 iters）：

函数	占比	计算类型
`MlasGemmQuantOperation`	74.51%	INT8 QGEMM kernel

函数	占比	计算类型
MlasComputeLogistic	9.80%	sigmoid 激活
QuickGelu	9.42%	GELU fusion
MlasReduceMinMaxF32Kernel	4.63%	min/max 归约
MlasQuantizeLinear	2.94%	量化
Im2col	1.35%	im2col 变换

全局指标：IPC = 0.48，每迭代 4.64 秒，L1-dcache miss rate 2.30%

2.3 ResNet50 推理热点

BB Rank	BB ID	执行占比	指令数	关键指令
1	85993	61.26%	16	vfmacc.vf, vle32.v, flw, addi, bgtu
2	85969	27.32%	16	vfmacc.vf, vle32.v, flw, addi, bgtu

- K 循环 BB 执行占比: 91.43%（BBV 加权）
- SGEMM 函数总占比: 93.38%

2.4 跨应用 K 循环一致性

应用	K 循环 BB 执行占比	BBV 数据来源
YOLO11n FP32	94.68%	QEMU BBV, VLEN=512, 3 iters
YOLO11n INT8	96.0%	QEMU BBV, VLEN=512, 10000 iters
ResNet50 FP32	91.43%	QEMU BBV, VLEN=512, 10 iters

三个应用 K 循环热点占比均在 91-96% 区间，确认 GEMM K 循环是跨应用的绝对性能瓶颈。

三、CX 指令延迟参考

基于 docs/reference/cx/instruction-constraints-and-latency.md：

指令	延迟	说明
flw	4	标量 float 加载
vle32.v	3	向量 float 加载
vle8.v	3	向量 byte 加载
vfmacc.vf	5	标量广播 FMA
vfmacc.vv	5	向量 FMA
vfmul.vv	5	向量乘法
vfmul.vf	5	标量广播乘法
vfdiv.vv	4-17	向量除法（不定周期）
vfmax.vf	3	标量广播 max
vfmin.vf	3	标量广播 min
vwmul.vx (SEW=8)	4+5 = 9	标量广播 widening 乘
vwmul.vv (SEW=8)	4	向量 widening 乘
vwadd.wv	4	Widening 累加
vwmaccu.vv	5	Widening MAC（vector）
vfcvt.x.f.v	4	float→int 转换

指令	延迟	说明
vncvt	4	窄化转换
vfredmax.vs	4	float max 归约
vfredmin.vs	4	float min 归约
vfmv.s.f	3	标量→向量元素 0
vfmv.f.s	3	向量元素 0→标量
vsetvli	2	VL 配置
vse8.v	4	byte 存储
vse32.v	4	float 存储
vfrsqrt7.v	4-17	近似倒数平方根（不定周期）
vfrec7.v	4-17	近似倒数（不定周期）

四、BBV 实测数据

4.1 FP32 SGEMM（MlasSgemmKernelRvv512）

数据项	值
K-loop BB 执行比例	**94.68%**
K-loop 每迭代指令数	16 条（4 flw + 2 vle32 + 4 vfmacc + 5 addi + 1 bgtu）
总 BB 执行次数	2,526,357,651
flw 总执行次数	9,567,360,000
vfmacc.vf 总执行次数	9,567,360,000
vle32.v 总执行次数	4,783,680,000

4.2 INT8 QGEMM（MlasQgemmKernelRvv512Impl）

数据项	值
K-loop BB 执行比例	**96.0%**
K-loop 每 PackedK group 指令数	30 条（4 lbu + 4 vle8 + 4 vwmulu + 4 vwaddu + 9 vsetvli + 5 addi/bne）
lbu 总执行次数	204,800,000
vle8.v 总执行次数	138,240,000
vwmulu.vx 总执行次数	81,920,000
vwaddu.wv 总执行次数	66,560,000
vsetvli 总执行次数	133,120,000

4.3 ResNet50 FP32 SGEMM

数据项	值
K-loop BB 执行比例（加权）	**91.43%**
SGEMM 函数总占比	**93.38%**
Top 2 K-loop BB 占比	**88.58%**（BB 85993: 61.26% + BB 85969: 27.32%）
K-loop 每迭代指令数	16 条（4 flw + 2 vle32 + 4 vfmacc + 2 addi + 2 bgtu + 2 其他）

4.4 YOLO INT8 算子 BBV

算子	核心循环 BB 指令数	执行次数	关键指令
ComputeLogistic	25 条/迭代	9,050	vle32 → vfmax → vfmin → vfmul → 9×vfmacc → vfdiv → vfadd → vfmax → vfmin → vse32
QuickGelu Alpha	7 条/迭代	10,136	vsetvli → vle32 → vfmul.vf → vse32
ReduceMinMax 主循环	18 条/迭代	3,114	4×vle32 → 4×vfmin → 4×vfmax
QuantizeLinear	10 条/迭代	52 BBs	vle32 → vfdiv → vfmax → vfmin → vfcvt → vmv → vadd → vncvt → vncvt → vse8

4.5 llama.cpp 量化算子 BBV

算子	.bb 文件大小	BB 数量	总执行次数	SEW	关键指令
gemm-q4_K-8x4-q8_K	1.95MB	3564	82,295,020	e8/e16/e32	vlse8 → vand → vsrl → vwmacc.vx → vwmacc.vv → vfcvt → vfmacc
gemv-q4_K-8x8-q8_K	401KB	3558	6,633,029	(标量回退)	j → generic 标量实现 (LLVM 22 bug)
quantize-q8_0-4x4	265KB	3249	3,922,390	e32/e16/e8	vle32 → vfabs → vfredmax → vfmul → vfcvt → vncvt → vsseg4e32
vec-dot-q5_0-q8_0	293KB	3487	2,723,590	e8/e16/e32	vle8 → vand → vsrl → vlmul_ext → vslideup → vlm → vmnand → vsub.mu → vwmul → vwredsum

K-loop 执行占比（gemm/vec-dot 类算子）：65-85%（基于 BBV 权重估算）

关键发现:

- gemv kernel 为 2 字节 trampoline 标量回退（LLVM 22 优化器 bug），无有效向量化
- vec-dot 掩码符号扩展（vlm+vmnand+vsub.mu）为 RVV 独有优势：3 条 vs x86 4 条 + 查表
- quantize vsseg4e32 段存储为 RVV 决定性优势：1 条 vs ARM NEON 288 条 lane 提取+store
- 量化点积三函数（vec-dot-q5_0/q6_K/q4_K）占硬件 perf 88.71%，均依赖 int8→int32 归约

五、与主流架构对比

5.1 扩展后性能定位（相对吞吐量）

指标	x86 VNNI	ARM SDOT	RVV 现状	RVV 扩展后
INT8 GEMM 吞吐	100%（基准）	80-100%	30-50%	85-100%
FP32 GEMM 吞吐	100%（基准）	90%	60%	85-100%
Lane-indexed 操作	有	有	无	有
饱和窄化	有	有	无	有
单指令水平归约	有	有	无	有

5.2 关键差距填补

差距类型	RVV 现状	提议方案	填补效果
INT8 分段规约	vwmulu+vwaddu 两步	方案 1: vsegdot.vv	达到 VNNI/SDOT 水平

差距类型	RVV 现状	提议方案	填补效果
Lane-indexed	仅 .vx 标量广播	方案 4: vfmacc.vv_lane / 方案 2: vdot_lane.vx	消除广播开销
矩阵外积	需 16 条 vfmacc	方案 5: vmulacc.vv（1 条）	减少 75% 指令
Clamp	2 条 vfmax+vfmin	方案 6: vclamp.vf	减少 50% clamp 指令
水平 min/max 归约	3 步序列	方案 7: vfmax.red/vfmin.red	达到 NEON64 水平
饱和窄化	2 步 vncvt（无饱和）	方案 3: vnarrow_sat	达到 SSE2/NEON 水平
ABS+归约 max	vfabs+vfredmax+vfmv.f.s 3 步	方案 12: vfabs_redmax	超过 NEON vmaxvq（仅归约）
Widening multiply-reduce	vwmul+vwredsum+vmv 3 步	方案 11: vwmulred.vs	接近 ARM vdot+vaddv
Nibble 解包	vand+vsrl 2 步	方案 15: vnibunpack.vv	达到 x86 PSHUFB pattern 水平
数据预取	完全缺失	方案 13: prefetch.v	达到 x86 prefetcht0/ARM PRFM 水平
Sum 归约	vfmv+vfredusum+vfmv.f.s 3 步	方案 16: vfadd.red.vs	达到 NEON64 vaddvq 水平
硬件 exp	~28 条多项式指令	方案 17: vfexp.v	达到 AVX512 ER vexp2ps 水平

5.3 跨应用累计收益估算

应用场景	最高收益方案	整体收益
llama.cpp 推理	方案 1 vsegdot.vv + LLVM bug 修复	+30-40%（Phase 1 完成）
llama.cpp 推理	方案 1+10+12 全部	+40-50%（累计）
YOLO11n FP32	方案 5 vmulacc.vv	44.6%
YOLO11n FP32	方案 4 vfmacc.vv_lane	29.1%
YOLO11n INT8	方案 1 vsegdot.vv	54%
ResNet50 FP32	方案 5 vmulacc.vv	44.55%
ResNet50 FP32	方案 4 vfmacc.vv_lane	22.86%
ONNXRT SGEMM	方案 5 vmulacc.vv (vmatmul.fp32)	77.5%（K-loop BB 内）
OSTrack ReduceMean	方案 16 vfadd.red.vs	归约 BB 内 67%
OSTrack Softmax	方案 16+7 归约优化	Pass 1+2 归约阶段 ~24%
OSTrack Softmax	方案 17 vfexp.v	exp BB 内 ~96%

5.4 RVV 独有优势（跨应用确认）

优势	受益算子	应用	优势描述
掩码符号扩展	vec-dot-q5_0	llama.cpp	vlm+vmnand+vsub.mu 3 条 vs x86 4 条+查表
段存储交织	quantize-q8_0-4x4	llama.cpp, YOLO	vsseg4e32 1 条 vs ARM NEON 288 条
可配置 VLEN	全部	全部	单一代码路径覆盖 128/256/512
LMUL 跨寄存器	quantize	llama.cpp, YOLO	m8 加载 32 元素 vs ARM 8 次加载
无序归约 vfredusum	ReduceMean, Softmax	OSTrack	硬件树形归约 vs ARM/AVX 软件 shuffle 序列
sigmoid-based exp	Softmax	OSTrack	复用验证多项式系数，精度 ~1e-7（已 QEMU 验证）
Horner vmull+vfadd	Softmax, ComputeLogistic	OSTrack, YOLO	正确多项式评估 vs vfmacc 非 Horner 形式（已发现 bug）